Chapter 8 – Data Visualization

IERG4320/IEMS5726 Data Science in Practice | Cheat Sheet

Introduction

数据可视化：用图形展示数据中的模式和趋势

Goals: explore, monitor, explain data.

Benefits: reveal patterns/trends, identify outliers, simplify complex info, aid decision-making.

When to visualize:

- Preprocessing: distribution, trends 看数据分布
- Analysis: model performance 看模型表现
- Interpretation: present findings 展示结果

Packages: `matplotlib.pyplot as plt`, `seaborn as sns`

Seaborn建在matplotlib之上，和pandas配合好

Preprocessing Visualizations

1. Histogram (Distribution)

直方图：看单个特征的数据分布

```
import seaborn as sns
import matplotlib.pyplot as plt
penguins = sns.load_dataset("penguins")
# Basic histogram
sns.displot(penguins, x="flipper_length_mm")
# Custom bins
sns.displot(penguins, x="flipper_length_mm",
            binwidth=3)
sns.displot(penguins, x="flipper_length_mm",
            bins=20)
# Colored by category
sns.displot(penguins, x="flipper_length_mm",
            hue="species")
# Stacked
sns.displot(penguins, x="flipper_length_mm",
  hue="species", multiple="stack")
# Side-by-side
sns.displot(penguins, x="flipper_length_mm",
  hue="sex", multiple="dodge")
plt.show()
```

2. Boxplot

箱线图：显示中位数、四分位数和异常值

Key: Median, Q1 (25%), Q3 (75%), IQR=Q3–Q1.

Whiskers: Q1–1.5×IQR to Q3+1.5×IQR. Beyond = outliers.

```
# Pandas approach
penguins.boxplot(column=["flipper_length_mm"])
# Conditional boxplot
penguins.boxplot(column=["flipper_length_mm"],
                 by=["species"])
# Seaborn approach
sns.boxplot(data=penguins,
  x="flipper_length_mm", y="sex",
  hue="species")
plt.show()
```

3. Scatter Plot

散点图：看两个特征的关系

```
# Matplotlib
plt.scatter(x, y)
plt.title("Title"); plt.xlabel("X")
plt.ylabel("Y"); plt.savefig("name")
# Seaborn (with hue grouping)
sns.scatterplot(x="total_bill", y="tip",
  data=tips, hue="smoker",
  hue_order=["Yes","No"])
plt.show()
```

4. Q-Q Plot

QQ图：检查数据是否服从某分布（如正态分布）

Points on 45-degree line = same distribution.

```
import statsmodels.api as sm
import pylab as py
data_pts = np.random.normal(0, 1, 100)
sm.qqplot(data_pts, line='45')
py.show()
```

5. Heatmap

热力图：用颜色表示矩阵中数值大小，常用于看特征相关性

```
glue = sns.load_dataset("glue").pivot(
  index="Model",columns="Task",values="Score")
sns.heatmap(glue)            # basic
sns.heatmap(glue, annot=True) # with numbers
plt.show()
```

6. Pair Plot

配对图：所有特征两两组合的散点图+对角线分布图

```
sns.pairplot(USAhousing)
plt.show()
```

Analysis Visualizations

1. ROC Curve

ROC曲线：FPR vs TPR，AUC越大模型越好

```
from sklearn.metrics import roc_curve, \
  roc_auc_score
fpr, tpr, _ = roc_curve(y_test, y_proba)
auc = roc_auc_score(y_test, y_proba)
plt.plot(fpr, tpr, label=f"AUC={auc:.3f}")
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(); plt.show()
```

2. Training Curve (ANN)

训练曲线：看loss是否收敛，train和val差距是否合理

```
plt.plot(iters, losses, label="Train")
plt.xlabel("Iterations")
plt.ylabel("Loss"); plt.show()
# Accuracy curve
plt.plot(iters, train_acc, label="Train")
plt.plot(iters, val_acc, label="Validation")
plt.legend(loc='best'); plt.show()
```

train和val差距大=过拟合; 都高=欠拟合

3. Predicted vs Actual

预测值vs真实值：点越接近对角线=模型越好

```
plt.scatter(true_value, predicted_value,
            c='crimson')
p1=max(max(predicted_value),max(true_value))
p2=min(min(predicted_value),min(true_value))
plt.plot([p1,p2],[p1,p2],'b-') # diagonal
plt.xlabel('True'); plt.ylabel('Pred')
plt.show()
```

4. Residual Plot

残差图：检查回归模型假设是否成立（残差应随机分布）

```
sns.residplot(x='Head_size',
  y='Brain_weight', data=data)
plt.show()
```

5. Calibration Plot

校准图：预测概率是否和实际频率一致

```
from sklearn.calibration import calibration_curve
prob_true, prob_pred = calibration_curve(
  y_test, y_probs, n_bins=10)
plt.plot(prob_pred, prob_true, marker='o')
plt.plot([0,1],[0,1],'--',color='gray')
plt.xlabel('Mean predicted probability')
plt.ylabel('Fraction of positives')
plt.show()
```

6. Learning Curve

学习曲线：看增加训练数据是否能提升模型

```
from sklearn.model_selection import learning_curve
sizes, train_sc, val_sc = learning_curve(
  estimator=LinearRegression(),
  X=df[features], y=df[target],
  train_sizes=[1,100,500,2000,5000,7654],
  cv=5, scoring='neg_mean_squared_error')
plt.plot(sizes,-train_sc.mean(axis=1),
         label='Train error')
plt.plot(sizes,-val_sc.mean(axis=1),
         label='Validation error')
plt.legend(); plt.show()
```

两条线差距很小=理想; 差距大=模型太简单或数据不够

Combining Multiple Plots

1. Subplots Grid

```
fig, axes = plt.subplots(nrows=2, ncols=2,
                         figsize=(12, 8))
axes[0,0].plot(x, y1); axes[0,0].set_title('A')
axes[0,1].plot(x, y2); axes[1,0].plot(x, y3)
plt.tight_layout(); plt.show()
```

2. Overlay on Same Axes

```
plt.plot(x, y1, 'b-', label='sin(x)')
plt.plot(x, y2, 'r--', label='cos(x)')
plt.legend(); plt.show()
```

3. Mixed Plot Types

```
fig, ((ax1,ax2),(ax3,ax4)) = plt.subplots(2,2)
ax1.bar(cats, vals)      # bar
ax2.scatter(sx, sy)      # scatter
ax3.hist(data, bins=15)  # histogram
ax4.pie(vals, labels=cats, autopct='%1.1f%%')
```

4. Shared Axes

```
fig, (ax1,ax2,ax3) = plt.subplots(1, 3,
  figsize=(15,5), sharey=True)
```

sharey=True让多个子图共享Y轴，便于比较

Text Visualization

Word Cloud

词云：词频越高，字体越大

```
from wordcloud import WordCloud
# From text
wc = WordCloud(width=800, height=400,
  background_color='white').generate(text)
plt.imshow(wc, interpolation='bilinear')
plt.axis('off'); plt.show()

# From frequency dict
word_freq = {'python':45, 'data':38, ...}
wc = WordCloud(width=800, height=400,
  background_color='lightblue', colormap='cool'
).generate_from_frequencies(word_freq)

# Multiple documents as subplots
fig, axes = plt.subplots(1, 3, figsize=(18,6))
for ax, (title, text) in zip(axes, texts.items()):
  wc = WordCloud(max_words=30).generate(text)
  ax.imshow(wc); ax.set_title(title)
  ax.axis('off')
```

Slope Chart & Sankey

Slope Chart: show changes over time / rankings.

斜率图：展示两个时间点之间的变化

Sankey Chart: visualize flow between nodes.

桑基图：展示数据流向（如能源流、供应链）

```
import plotly.graph_objects as go
fig = go.Figure(data=[go.Sankey(
  node=dict(pad=15, thickness=20,
    label=["SrcA","SrcB","ProcX","Out1"]),
  link=dict(source=[0,0,1],target=[2,3,2],
            value=[8,4,2]))])
fig.show()
```

Image Parsing & Model Architecture

Object Detection (Mask R-CNN)

目标检测：识别图片中的物体位置和类别

```
import torchvision
model = torchvision.models.detection\
  .maskrcnn_resnet50_fpn(weights=...)
model.eval()
with torch.no_grad():
  predictions = model([image_tensor])
# predictions: boxes, scores, labels, masks
```

torchinfo – Visualize Model Layers

查看神经网络每层的输出形状和参数数量

```
from torchinfo import summary
summary(model, input_size=(64, 28*28))
# Shows: layer type, output shape, param count
# Total params, trainable params, size
```

GeoPandas & Map Visualization

GeoPandas：在pandas基础上加地理空间数据支持

Loading Geospatial Data

```
import geopandas as gpd
# Load from file
gdf = gpd.read_file("example.shp")    # shapefile
gdf = gpd.read_file("example.geojson") # GeoJSON
gdf = gpd.read_file("example.gpkg")   # GeoPackage
# Load from URL
url = "https://naciscdn.org/naturalearth/..."
world = gpd.read_file(url)
```

Shapefile=.shp+.shx+.dbf; GeoJSON=JSON格式; GPKG=SQLite

Geometry Attributes & Methods

Attributes: `area`, `bounds`, `total_bounds`, `geom_type`, `is_valid`

Methods: `distance()`, `centroid`, `to_crs()`, `plot()`

Choropleth Map

分级统计图：用颜色深浅表示数值大小

```
# Simple world map
world.plot(figsize=(10,6))
# Choropleth with color by column
ax = world["geometry"].boundary.plot(
  figsize=(20,16))
merged_df.plot(column="Discipline", ax=ax,
  cmap='OrRd', legend=True,
  legend_kwds={"label":"Participation",
    "orientation":"horizontal"})
ax.set_title("Title"); plt.show()
```

Layered Map

叠加多层地图：边界+区域+点+线

```
# Base layer: county boundaries
ax = counties.plot(figsize=(10,10),
  color='none', edgecolor='gainsboro')
# Add layers
wild_lands.plot(color='lightgreen', ax=ax)
campsites.plot(color='maroon',markersize=2,
               ax=ax)
trails.plot(color='black',markersize=1,ax=ax)
plt.show()
```

Visualization Best Practices

可视化7条建议

1. **Label directly** on the chart 直接在图上标注
2. **Repeat units** 重复标注单位
3. **Move axis ticks** where needed 刻度放在需要的地方
4. **Don't center-align** text 不要居中对齐文字
5. **Don't rotate** text 不要旋转文字
6. **Use text outline** for readability 用文字描边提高可读性
7. **Choose suitable number format** 选合适的数字格式

Tools: Matplotlib, Seaborn, Plotly, WordCloud, GeoPandas, torchinfo.
Business: Excel, Google Data Studio, Tableau, Power BI.